

PITS-MRAS: Complete Implementation Plan for Claude Code

Handoff Document — Read This First This document is a complete, self-contained specification for building out the PITS-MRAS repository from its current scaffold-only state into a working Python framework. Read every section before touching a single file. The mathematical identities in §3 are load-bearing; implement them incorrectly and the theoretical guarantees collapse. Work through phases in order — each phase depends on the previous one.

§0 • Ground Truth: What Currently Exists

Clone the repo first and verify:

```
git clone https://github.com/danielsimonjr/PITS-MRAS.git
cd PITS-MRAS
find . -type f -not -path "./.git/*" | sort
```

You will find: `src/README.md`, `tests/README.md`, `examples/README.md` — **all three implementation directories are empty stubs**. The only substantive content is in `docs/`. Every `.py` file in this plan is net-new. The mathematical specification in `docs/PITS-MRAS — Physics-Informed...md` (1,543 lines) is your source of truth for the existing design; this plan extends it with RL-optimal control connections derived from the companion reference document.

§1 • What We Are Building and Why

PITS-MRAS merges three paradigms:

1. **Physics-Informed Neural Networks (PINNs)** — the PITNN encoder enforces conservation laws via port-Hamiltonian structure (Hamiltonian H_θ , skew-symmetric interconnection J , positive-definite dissipation $R_\theta = L^\top L$).
2. **Time-Series Deep Learning** — LSTM encoder + multi-head physics-informed attention (temporal, physical, error-driven) processes the state/control history.

3. **Model-Reference Adaptive Systems (MRAS)** — a linear reference model $\dot{x}_m = A_m x_m + B_m r$ (Hurwitz A_m) defines the desired closed-loop behavior; a Lyapunov function $V(e, \tilde{\theta}, \tilde{\theta}_c) = e^T P e + \dots$ guarantees tracking-error convergence.

The core upgrade this plan implements is the formal unification with RL/optimal control theory through ten mathematical identities. The three most important (highest rigor, lowest implementation cost) are:

- **Identity 1 (Lyapunov = Value Function):** For $V(e) = e^T P e$ and the Lyapunov equation $A_m^T P + P A_m = -Q$, this IS the policy-evaluation step of Kleinman's 1968 algorithm (i.e., one step of policy iteration on the CARE). Adding a learned critic with an Integral RL Bellman error loss upgrades MRAS from static one-step evaluation to full iterative policy iteration.
- **Identity 2 (Costate = Critic Gradient):** By Pontryagin's Minimum Principle, the costate $\lambda(t) = \partial V / \partial e$. The optimal control is $u^* = -R^{-1} B^T \lambda \dot{Y} = -R^{-1} B^T \nabla \hat{V}$. Making the action head the autodiff gradient of a scalar value head enforces this identity *by construction* (same trick PINNs use for exact derivatives).
- **Identity 3 (CLF-CBF-QP Safety Filter):** The informal " $\dot{V} < 0 \rightarrow$ apply; else emergency backup" in the existing flow diagram has a rigorous closed-form replacement: project u_{nominal} onto the safe half-space defined by the CBF constraint. For a single constraint (an ellipsoidal tracking-error bound), this is one line of math with no external QP solver.

The remaining seven connections are implemented as optional modules that can be enabled via configuration flags without breaking the baseline.

§2 · Updated Dependencies

Replace `requirements.txt` entirely with the following:

```
# Core scientific stack
numpy>=1.24.0
scipy>=1.10.0
torch>=2.1.0
torchvision>=0.16.0

# Physics / control utilities
control>=0.9.4          # Python Control Systems Library (Riccati, Lyapunov solv

# Training infrastructure
matplotlib>=3.7.0
```

```

seaborn>=0.12.0
pandas>=2.0.0
tqdm>=4.65.0
pyyaml>=6.0.0
tensorboard>=2.13.0
wandb>=0.15.0

# Testing and code quality
pytest>=7.4.0
pytest-cov>=4.1.0
black>=23.0.0
flake8>=6.0.0
mypy>=1.4.0
isort>=5.12.0

# Optional: CBF-QP with multiple constraints
# (not required for single-constraint closed-form – only needed if you add >1 CBF
# cvxpy>=1.3.0

```

Also update `setup.py` to reflect the new package structure:

```

# setup.py
from setuptools import setup, find_packages

setup(
    name="pits_mras",
    version="0.1.0",
    packages=find_packages(where="src"),
    package_dir={"": "src"},
    python_requires=">=3.9",
    install_requires=[
        "numpy>=1.24.0",
        "scipy>=1.10.0",
        "torch>=2.1.0",
        "control>=0.9.4",
        "matplotlib>=3.7.0",
        "pandas>=2.0.0",
        "tqdm>=4.65.0",
        "pyyaml>=6.0.0",
    ],
    extras_require={
        "dev": ["pytest>=7.4.0", "pytest-cov>=4.1.0", "black>=23.0.0",
                "flake8>=6.0.0", "mypy>=1.4.0", "isort>=5.12.0"],
        "logging": ["tensorboard>=2.13.0", "wandb>=0.15.0"],
    },
)

```

§3 · Mathematical Reference: Identities Claude Code Must Preserve

This section defines every formula that appears in the code. Do not approximate or simplify these — they are the correctness contract.

3.1 Port-Hamiltonian Dynamics Decoder

The PITNN's physics decoder produces dynamics by separating conservative and dissipative components, inspired by port-Hamiltonian theory:

```
Conservative:    f_cons = J(q) · ∇_{[q;p]} H_θ(q, p)           where J = -JT (skew-sy
Dissipative:     f_diss = -R_θ(q) · q̇                        where R_θ = LT · L ≥ 0
Control input:   f_ctrl = B(x_p) · u
Temporal corr:   f_corr = W_corr · c_t + b_corr              (residual from attent
Total:           f_θ = f_cons + f_diss + f_ctrl + f_corr
```

The Hamiltonian $H_\theta(q, p)$ is a scalar neural network with outputs constrained to be positive (via softplus or exp output activation). The matrix $L_\theta(q)$ is a lower- triangular neural network output; $R_\theta = L^T \cdot L$ guarantees positive semi-definiteness automatically.

3.2 MRAS Lyapunov / Value Function (Identity 1)

The tracking error is $e(t) = y_p(t) - y_m(t)$. The classical MRAS Lyapunov function is:

$$V(e, \tilde{\theta}, \tilde{\theta}_c) = e^T P e + \tilde{\theta}^T \Gamma_\theta^{-1} \tilde{\theta} + \tilde{\theta}_c^T \Gamma_c^{-1} \tilde{\theta}_c$$

where $P > 0$ solves the discrete-time Lyapunov equation $A_m^T P + P A_m = -Q$.

The identity: $V(e) = e^T P e$ IS the LQR value function for the tracking-error system $\dot{e} = A_m e + \text{perturbation}$, with cost $\int (e^T Q e + u^T R u) dt$. The matrix equation $A_m^T P + P A_m = -Q$ is Kleinman's policy-evaluation step: given a fixed policy K (so $A_{cl} = A_m - BK$), solve the closed-loop Lyapunov equation for P , then improve via $K \leftarrow R^{-1} B^T P$. This implementation adds an **Integral RL (IRL) critic** that learns $\hat{P}$ from data without knowing A :

```
IRL Bellman error:    δ_IRL(t) = ∫_{t-T}^t (e^T Q e + u^T R u) dt - [V̂(e(t)) - V̂(e(
IRL loss:             L_IRL = ½ · E[δ_IRL²]
```

The key property (Vrabie & Lewis 2009, Lemma 1): the IRL Bellman equation does NOT contain the drift matrix A , so policy evaluation is model-free.

3.3 Costate Head (Identity 2)

Pontryagin's Minimum Principle: $\lambda(t) = \partial V / \partial e$ (costate = value gradient). The optimal control is:

$$u^* = -R^{-1} B^T \lambda(t) = -R^{-1} B^T \nabla_e \hat{V}(e)$$

In the LQR limit $\hat{V} = e^T P e$, this gives $u^* = -R^{-1} B^T P e = -K e$ (the LQR gain), confirming the identity. The architecture enforces this **by construction**: the action head computes $\nabla_e \hat{V}$ via `torch.autograd.grad(V_hat, e)`, then multiplies by $-R_{\text{inv}} @ B.T$. Never implement u as an independent neural network if the costate identity is to hold — it must be the analytic gradient of $\hat{V}$.

Two consistency losses reinforce this:

```
Gradient consistency:  L_costate = ||λ̂(t) - ∇_e V̂(e(t))||²
Adjoint dynamics:      L_adjoint = ||λ̇(t) + ∂H/∂e||²    evaluated along trajectory
```

3.4 CLF-CBF-QP Safety Filter (Identity 3)

For the affine system $\dot{e} = f(e) + g(e) \cdot u$, define the CBF as:

$$h(e) = c - e^T P e \quad (\text{safe when tracking error is inside the ellipsoid } e^T P e \leq c)$$

The CBF constraint requires:

$$L_f h(e) + L_g h(e) \cdot u \geq -\gamma h(e)$$

where $L_f h = -2e^T P A_m e$ and $L_g h = -2e^T P B$ (Lie derivatives along reference model and control directions). The **closed-form single-constraint solution**:

```
a = L_f_h + (L_g_h @ u_nom).squeeze() + gamma * h_e
b = L_g_h # shape [control_dim]

if a >= 0:
    u_safe = u_nom # constraint already satisfied, no modification needed
else:
    # Minimum-norm projection onto the safe half-space
    correction = (a / (b @ b)) * b # scalar correction along b direction
    u_safe = u_nom - correction
```

No external QP solver is needed for a single constraint. Use `cvxpy` only if you add more

than one CBF constraint later.

3.5 HJB Residual Loss (Identity 8)

The Hamilton-Jacobi-Bellman PDE at the optimum:

$$0 = \ell(e, u^*) + \nabla_e \hat{V} \cdot (A_m e + B u^* + f_{\text{corr}}) \\ = e^T Q e + (u^*)^T R u^* + \nabla_e \hat{V} \cdot A_m e + \nabla_e \hat{V} \cdot B u^* + \nabla_e \hat{V} \cdot f_{\text{corr}}$$

With $u^* = -R_{\text{inv}} B^T \nabla_e \hat{V}$ (from the costate head), the term $\nabla_e \hat{V} \cdot B u^* = -\nabla_e \hat{V}^T B R_{\text{inv}} B^T \nabla_e \hat{V} \leq 0$ (energy-dissipating). The loss is:

$$L_{\text{HJB}} = \left\| e^T Q e + (u^*)^T R u^* + \nabla_e \hat{V} \cdot (A_m e + B u^* + f_{\text{corr}}) \right\|^2$$

⚠ Calibration note: Start with weight $\lambda_{\text{HJB}} = 0.01$. Published experiments (HJBPPPO 2023) found this does not consistently improve over the baseline across all environments; treat it as a regularizer and tune per task.

3.6 Updated Adaptation Law (DPG Identity, Identity 4)

The classic MRAS update $\dot{\theta}_c = -\Gamma_c (\nabla L_{\text{total}} + \gamma_{\text{MRAS}} e \phi_c)$ is a deterministic policy gradient (Silver et al. 2014) with a fixed critic surrogate $e^T P e$. The upgrade replaces the surrogate with the learned critic gradient:

$$\dot{\theta}_c = -\Gamma_c \left[\nabla_{\theta_c} L_{\text{total}} + \phi_c \cdot \nabla_a \hat{Q}(e, u) |_{a=u} \right]$$

where $\nabla_a \hat{Q}$ reduces to $P e$ (up to factor 2) in the LQR limit, confirming backward compatibility. Implement this as a custom optimizer step that adds the DPG term after the standard gradient step.

§4 · Phase 1 — Foundation Layer

Create the package scaffolding and utility modules. **No neural networks yet** — only pure-math utilities that the network modules will import.

4.1 Package Init Files

Create `src/pits_mras/__init__.py`:

```
"""
PITS-MRAS: Physics-Informed Time-Series Model-Reference Adaptive Systems.
```

A unified framework merging Physics-Informed Neural Networks (PINNs), Time-Series Deep Learning, and Model-Reference Adaptive Control (MRAS).

Mathematical foundation: The MRAS Lyapunov function $V(e)=e^T P e$ is the LQR value function for the tracking-error system; policy iteration on the CARE (Kleinman 1968) is the formal backbone; IRL (Vrabie & Lewis 2009) makes it model-free. The port-Hamiltonian decoder makes H_θ a storage/value function (passivity = L2-gain), and the costate $\lambda=\nabla V$ is enforced architecturally.

"""

```
from pits_mras.models.pitnn import PITNN
from pits_mras.models.critic import QuadraticCritic
from pits_mras.controllers.mras import MRASController
from pits_mras.controllers.reference_models import LinearReferenceModel
from pits_mras.controllers.safety import CLFCBFSafetyFilter
from pits_mras.training.pretrain import pretrain_pitnn
from pits_mras.training.cotrain import cotraining_loop
from pits_mras.inference.realtime import RealtimeInferenceEngine

__version__ = "0.1.0"
__all__ = [
    "PITNN", "QuadraticCritic", "MRASController", "LinearReferenceModel",
    "CLFCBFSafetyFilter", "pretrain_pitnn", "cotraining_loop",
    "RealtimeInferenceEngine",
]
```

Create empty `__init__.py` files (no imports yet, just docstrings) in:

- `src/pits_mras/models/__init__.py`
- `src/pits_mras/controllers/__init__.py`
- `src/pits_mras/losses/__init__.py`
- `src/pits_mras/training/__init__.py`
- `src/pits_mras/inference/__init__.py`
- `src/pits_mras/utils/__init__.py`

4.2 Configuration System

Create `src/pits_mras/config.py`. This is a dataclass-based config that maps directly to the hyperparameters in the existing algorithm specs:

"""

Centralized configuration for PITS-MRAS.

All hyperparameters from the technical specification and RL extensions live here.

"""

```
from dataclasses import dataclass, field
```

```
from typing import List, Optional
```

```
import yaml
```

```
import torch
```

```
@dataclass
```

```
class NetworkConfig:
```

```
    """Architecture hyperparameters for PITNN."""
```

```
    input_dim: int = 10          # state + control dimension
```

```
    hidden_dim: int = 128
```

```
    output_dim: int = 4          # dynamics prediction dimension
```

```
    lstm_layers: int = 2
```

```
    attention_heads: int = 4
```

```
    memory_horizon: int = 50     # T: time steps of history to retain
```

```
    embedding_dim: int = 64
```

```
@dataclass
```

```
class PhysicsConfig:
```

```
    """Port-Hamiltonian decoder dimensions."""
```

```
    n_generalized_coords: int = 2 # n_q (positions)
```

```
    hamiltonian_hidden: int = 64   # width of H_θ network
```

```
    dissipation_hidden: int = 32   # width of L_θ network
```

```
    use_position_dependent_J: bool = False # set True for nonholonomic systems
```

```
@dataclass
```

```
class MRASConfig:
```

```
    """Classical MRAS and new IRL/actor-critic parameters."""
```

```
    state_dim: int = 4            # dimension of tracking error e
```

```
    control_dim: int = 2          # dimension of control input u
```

```
    # Reference model:  $\dot{x}_m = A_m x_m + B_m r$ . Supply as nested lists.
```

```
    A_m: Optional[List[List[float]]] = None # Hurwitz matrix, shape [state_dim,
```

```
    B_m: Optional[List[List[float]]] = None # shape [state_dim, control_dim]
```

```
    C_m: Optional[List[List[float]]] = None # shape [output_dim, state_dim]
```

```
    # LQR cost matrices
```

```
    Q_cost: Optional[List[List[float]]] = None # tracking cost, shape [state_di
```

```
    R_cost: Optional[List[List[float]]] = None # control cost, shape [control_d
```

```
    # Adaptation gains
```

```
    gamma_mras: float = 0.1       # classical MRAS adaptation rate
```

```
    adapt_rate_theta: float = 1e-4 # plant model learning rate
```

```
    adapt_rate_controller: float = 1e-3 # controller learning rate
```

```
    # IRL parameters (Connection 1)
```

```

    irl_window_size: int = 50          # T for IRL Bellman integral window
    use_irl_critic: bool = True        # enable Integral RL critic update

@dataclass
class SafetyConfig:
    """CLF-CBF-QP safety filter (Connection 3/6)."""
    enable_cbf: bool = True
    safety_margin: float = 10.0        #  $c$  in  $h(e) = c - e^T P e$ 
    cbf_decay_rate: float = 1.0        #  $\gamma$  in the CBF constraint

@dataclass
class LossConfig:
    """Loss weights for the unified total loss."""
    lambda_physics: float = 1.0
    lambda_temporal: float = 0.5
    lambda_stability: float = 2.0
    lambda_data: float = 1.0
    lambda_irl: float = 1.0            # IRL Bellman error weight
    lambda_hjb: float = 0.01           # HJB residual weight (tune carefully, see §
    lambda_costate: float = 0.1         # Costate consistency weight
    lambda_adjoint: float = 0.05        # Adjoint dynamics residual weight
    # Physics sub-weights
    lambda_energy: float = 1.0
    lambda_pde: float = 1.0
    lambda_bc: float = 0.5
    lambda_sym: float = 0.2
    # Temporal sub-weights
    alpha_attn: float = 0.1             # attention entropy regularization
    alpha_smooth: float = 0.05          # temporal smoothness
    # Stability sub-weights
    mu_lyap: float = 0.01               # exponential decay rate in  $L_{Lyap}$ 
    beta_param: float = 1e-4            # parameter boundedness regularization
    lambda_delta_u: float = 0.01        # control rate penalty

@dataclass
class TrainingConfig:
    """Training schedule parameters matching Algorithm 2 and Algorithm 3."""
    # Pre-training (Algorithm 2)
    pretrain_epochs: int = 5000
    pretrain_batch_size: int = 64
    pretrain_lr: float = 1e-3
    stage1_epochs: int = 1000           # physics-only
    stage2_epochs: int = 2000           # data-physics balance
    # Co-training (Algorithm 3)

```

```

n_episodes: int = 1000
sim_duration: float = 10.0          # T_sim in seconds
dt: float = 0.01                    # Δt in seconds
# General
device: str = "cuda" if torch.cuda.is_available() else "cpu"
seed: int = 42
log_every: int = 100
checkpoint_every: int = 500

@dataclass
class PITSMRASConfig:
    """Master configuration – the single object passed to all components."""
    network: NetworkConfig = field(default_factory=NetworkConfig)
    physics: PhysicsConfig = field(default_factory=PhysicsConfig)
    mras: MRASConfig = field(default_factory=MRASConfig)
    safety: SafetyConfig = field(default_factory=SafetyConfig)
    losses: LossConfig = field(default_factory=LossConfig)
    training: TrainingConfig = field(default_factory=TrainingConfig)

    @classmethod
    def from_yaml(cls, path: str) -> "PITSMRASConfig":
        with open(path) as f:
            d = yaml.safe_load(f)
            # Recursively build nested dataclasses from dict
            cfg = cls()
            for key, val in d.items():
                if hasattr(cfg, key) and isinstance(val, dict):
                    sub = getattr(cfg, key)
                    for k, v in val.items():
                        setattr(sub, k, v)
            return cfg

    def to_yaml(self, path: str) -> None:
        import dataclasses
        with open(path, "w") as f:
            yaml.dump(dataclasses.asdict(self), f, default_flow_style=False)

```

4.3 Lyapunov Utilities

Create `src/pits_mras/utils/lyapunov.py`. This is the mathematical engine for all P-matrix computations:

```

"""
Lyapunov and Riccati equation solvers.

```

Key identities (see §3.2):

- $A_m^T P + P A_m = -Q$ is both the MRAS Lyapunov equation AND Kleinman's policy-evaluation step for the tracking-error LQR.
- `scipy.linalg.solve_continuous_lyapunov` solves $A^T P + P A = -Q$.
- `scipy.linalg.solve_continuous_are` solves the full CARE for policy improvement.

```
"""
import numpy as np
import torch
from torch import Tensor
from scipy.linalg import solve_continuous_lyapunov, solve_continuous_are
from typing import Tuple, Optional

def solve_lyapunov(A_m: np.ndarray, Q: np.ndarray) -> np.ndarray:
    """
    Solve  $A_m^T P + P A_m = -Q$  for  $P > 0$ .
     $A_m$  must be Hurwitz (all eigenvalues have negative real parts).
    Returns  $P$  as numpy array with same shape as  $Q$ .
    """
    P = solve_continuous_lyapunov(A_m.T, -Q)
    eigvals = np.linalg.eigvalsh(P)
    if np.min(eigvals) <= 0:
        raise ValueError(
            f"P is not positive definite (min eigenvalue = {np.min(eigvals):.4e})"
            "Check that A_m is Hurwitz."
        )
    return P
```

```
def kleinman_iteration(
    A: np.ndarray,
    B: np.ndarray,
    Q: np.ndarray,
    R: np.ndarray,
    K_init: Optional[np.ndarray] = None,
    max_iter: int = 100,
    tol: float = 1e-10,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Kleinman's policy iteration (1968) for the CARE.
    Alternates between:
        Step 1 (policy evaluation): solve  $(A-BK)^T P + P(A-BK) + Q + K^T R K = 0$ 
        Step 2 (policy improvement):  $K \leftarrow R^{-1} B^T P$ 

    Returns  $(P_{\text{star}}, K_{\text{star}})$  at convergence.
    """
    n = A.shape[0]
```

```

K = K_init if K_init is not None else np.zeros((R.shape[0], n))
R_inv = np.linalg.inv(R)

for i in range(max_iter):
    A_cl = A - B @ K
    try:
        P = solve_continuous_lyapunov(A_cl.T, -(Q + K.T @ R @ K))
    except np.linalg.LinAlgError:
        raise RuntimeError(
            f"Lyapunov solve failed at iteration {i}. "
            "Closed-loop A_cl may not be Hurwitz – check initial K."
        )
    K_new = R_inv @ B.T @ P
    delta = np.linalg.norm(K_new - K, ord="fro")
    K = K_new
    if delta < tol:
        return P, K

raise RuntimeError(
    f"Kleinman iteration did not converge in {max_iter} steps "
    f"(final  $\Delta\|K\|_F = \{delta:.4e\})$ ."
)

def solve_care(
    A: np.ndarray, B: np.ndarray, Q: np.ndarray, R: np.ndarray
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Solve the Continuous Algebraic Riccati Equation directly via scipy.
    Returns (P_star, K_star) where K_star = R-1BTP_star.
    """
    P = solve_continuous_are(A, B, Q, R)
    K = np.linalg.solve(R, B.T @ P)
    return P, K

def check_hurwitz(A: np.ndarray, tol: float = 1e-6) -> bool:
    """Return True if all eigenvalues of A have negative real parts."""
    return bool(np.all(np.real(np.linalg.eigvals(A)) < -tol))

def lyapunov_derivative(
    e: Tensor, P: Tensor, A_m: Tensor, B: Tensor, u: Tensor
) -> Tensor:
    """
    Compute  $\dot{V} = 2e^T P (A_m e + B u)$  analytically.
    For a purely linear error dynamics  $\dot{e} = A_m e + B u$ , this is exact.
    """

```

```

Shapes: e [batch, n], P [n, n], A_m [n, n], B [n, m], u [batch, m].
Returns V_dot of shape [batch].
"""
e_dot_approx = e @ A_m.T + u @ B.T          # [batch, n]
Pe = e @ P                                  # [batch, n]
V_dot = 2.0 * (Pe * e_dot_approx).sum(dim=-1) # [batch]
return V_dot

def quadratic_basis(e: Tensor) -> Tensor:
    """
    Compute the upper-triangular Kronecker product basis for quadratic forms.
    For a linear critic  $\hat{V}(e) = W^T \phi(e)$ , this basis gives  $\hat{V} = e^T \hat{P} e$  exactly.

    For e of shape [batch, n], returns  $\phi(e)$  of shape [batch, n*(n+1)//2].
    Ordering:  $[e_1^2, e_1 \cdot e_2, e_1 \cdot e_3, \dots, e_2^2, e_2 \cdot e_3, \dots, e_n^2]$ .
    """
    batch, n = e.shape
    pairs = []
    for i in range(n):
        for j in range(i, n):
            pairs.append(e[:, i] * e[:, j])
    return torch.stack(pairs, dim=1) # [batch, n*(n+1)//2]

```

4.4 Port-Hamiltonian Utilities

Create `src/pits_mras/utils/hamiltonian.py`:

```

"""
Port-Hamiltonian system utilities.

A port-Hamiltonian system satisfies:
 $\dot{x} = (J(q) - R(q)) \nabla H(q,p) + g(x) u$ 
 $y = g(x)^T \nabla H(q,p)$ 

with  $J = -J^T$  (skew-symmetric, energy-conserving) and  $R = L^T L \geq 0$  (dissipative).

The storage function  $H$  satisfies  $\dot{H} = -\nabla H^T R \nabla H + y^T u \leq y^T u$  (passivity inequality)
which is the continuous-time analog of the RL Bellman inequality.
"""
import torch
from torch import Tensor
import torch.nn.functional as F

def make_skew_symmetric(raw: Tensor) -> Tensor:

```

```

    """
    Convert a [batch, n, n] raw tensor to a skew-symmetric matrix.
    Uses  $J = (raw - raw^T) / 2$ .
    """
    return (raw - raw.transpose(-1, -2)) / 2.0

def make_positive_definite(L: Tensor, epsilon: float = 1e-6) -> Tensor:
    """
    Given L of shape [batch, n, n] (lower-triangular network output),
    compute  $R = L^T L + \epsilon I > 0$ .
    This guarantees positive definiteness for any L.
    """
    LtL = torch.bmm(L.transpose(-1, -2), L)
    eye = epsilon * torch.eye(L.shape[-1], device=L.device, dtype=L.dtype)
    return LtL + eye.unsqueeze(0)

def port_hamiltonian_energy_loss(
    H_pred: Tensor,          # predicted Hamiltonian values [batch]
    dH_dt: Tensor,          # time derivative of H [batch]
    P_control: Tensor,       # control power  $y^T u$  [batch]
    P_diss: Tensor,          # dissipated power  $\nabla H^T R \nabla H \geq 0$  [batch]
) -> Tensor:
    """
    Enforce the port-Hamiltonian dissipation inequality:
         $dH/dt = P\_control - P\_dissipation$ 
    As a loss:  $\| dH/dt - P\_control + P\_diss \| ^2$ 
    """
    residual = dH_dt - P_control + P_diss
    return (residual ** 2).mean()

def hamiltonian_positivity_loss(H: Tensor) -> Tensor:
    """
    Enforce  $H > 0$  everywhere (H is an energy function → must be non-negative).
    Loss =  $\text{mean}(\text{ReLU}(-H))$ .
    """
    return F.relu(-H).mean()

```

4.5 Persistence of Excitation Monitor

Create `src/pits_mras/utils/pe_monitor.py`:

```

"""
Persistence of Excitation (PE) monitor.

```

IRL/ADP parameter convergence requires PE of the regressor $\phi_c(e, t)$.
This module checks the PE condition and, if not met, adds a small
probing signal to the control input to satisfy it.

PE condition: $\exists \delta, T > 0$ such that $\int_{t}^{t+T} \phi(\tau) \phi(\tau)^T d\tau \geq \delta I \quad \forall t$.

"""

```
import torch
from torch import Tensor
from collections import deque
```

```
class PEMonitor:
```

```
    """
```

```
    Monitors the minimum eigenvalue of the regressor Gram matrix over a sliding w
    Issues a warning and injects probing noise if the PE condition is violated.
```

```
    """
```

```
    def __init__(
        self,
        regressor_dim: int,
        window_size: int = 200,
        pe_threshold: float = 1e-3,
        noise_std: float = 0.01,
    ):
        self.regressor_dim = regressor_dim
        self.window_size = window_size
        self.pe_threshold = pe_threshold
        self.noise_std = noise_std
        self._buffer: deque = deque(maxlen=window_size)
```

```
    def update(self, phi: Tensor) -> None:
        """Add a new regressor vector (shape [n]) to the buffer."""
        self._buffer.append(phi.detach().cpu())
```

```
    def is_pe_satisfied(self) -> bool:
        """Return True if the PE condition holds for the current window."""
        if len(self._buffer) < self.window_size:
            return False
        Phi = torch.stack(list(self._buffer)) # [window_size, n]
        gram = (Phi.T @ Phi) / self.window_size
        min_eig = torch.linalg.eigvalsh(gram).min().item()
        return min_eig > self.pe_threshold
```

```
    def get_probing_noise(self, control_dim: int, device: str = "cpu") -> Tensor:
        """Return small probing noise to add to u when PE is not satisfied."""
        return torch.randn(control_dim, device=device) * self.noise_std
```

§5 · Phase 2 — Neural Network Models

5.1 Physics-Informed Attention Module

Create `src/pits_mras/models/attention.py`:

```
"""
Multi-head physics-informed attention combining three complementary attention typ
    1. Temporal attention: when in the past is relevant?
    2. Physical attention: which physical quantities (position, velocity, force) ma
    3. Error-driven attention: which past moments had similar tracking-error patter

These are gated together via a learned softmax weighting.
"""
import torch
import torch.nn as nn
from torch import Tensor
import torch.nn.functional as F
import math

class PhysicsInformedAttention(nn.Module):
    """
    Three-headed attention module for the PITNN encoder.

    Inputs:
        H_enc: [batch, T, d_k] - encoded hidden states from LSTM
        e_hist: [batch, T, e_dim] - tracking error history
        x_p: [batch, n_state] - current plant state
        e_curr: [batch, e_dim] - current tracking error

    Output:
        context: [batch, d_k] - weighted context vector c_t
        alpha: [batch, T] - combined attention weights
    """

    def __init__(
        self,
        d_k: int,          # key/query dimension
        e_dim: int,         # tracking error dimension
        n_state: int,       # full state dimension
        n_heads: int = 4,
    ):
```

```

super().__init__()
self.d_k = d_k
self.n_heads = n_heads

# Temporal attention: standard scaled dot-product
self.W_Q = nn.Linear(d_k, d_k, bias=False)
self.W_K = nn.Linear(d_k, d_k, bias=False)

# Physical attention: maps state/velocity/force to importance weights
self.W_phys = nn.Linear(n_state * 3, 1) # x_p, x_p_dot, u concatenated

# Error-driven attention: cosine similarity between current and past error
self.W_e = nn.Linear(e_dim, e_dim, bias=False) # optional projection

# Gating network: which attention type to trust?
gate_input_dim = d_k + e_dim + n_state
self.W_gate = nn.Linear(gate_input_dim, 3) # 3-way softmax

def forward(
    self,
    H_enc: Tensor, # [batch, T, d_k]
    e_hist: Tensor, # [batch, T, e_dim]
    x_p: Tensor, # [batch, n_state]
    e_curr: Tensor, # [batch, e_dim]
    x_p_dot: Tensor, # [batch, n_state]
    u_curr: Tensor, # [batch, control_dim]
) -> tuple[Tensor, Tensor]:
    batch, T, _ = H_enc.shape
    h_t = H_enc[:, -1, :] # [batch, d_k] - current hidden state

    # --- 1. Temporal attention (scaled dot-product) ---
    Q = self.W_Q(h_t).unsqueeze(1) # [batch, 1, d_k]
    K = self.W_K(H_enc) # [batch, T, d_k]
    scores_time = (Q @ K.transpose(-1, -2)).squeeze(1) / math.sqrt(self.d_k)
    alpha_time = F.softmax(scores_time, dim=-1) # [batch, T]

    # --- 2. Physical attention ---
    # Concatenate state, velocity, and control as physical descriptor
    phys_feat = torch.cat([x_p, x_p_dot, u_curr], dim=-1) # [batch, n_state*3]
    # Broadcast to each time step (physical attention is time-independent here)
    scores_phys = self.W_phys(phys_feat).expand(batch, T) # [batch, T]
    alpha_phys = F.softmax(scores_phys, dim=-1) # [batch, T]

    # --- 3. Error-driven attention (cosine similarity) ---
    e_proj = self.W_e(e_curr).unsqueeze(1) # [batch, 1, e_dim]
    e_hist_norm = F.normalize(e_hist, dim=-1) # [batch, T, e_dim]
    e_curr_norm = F.normalize(e_proj, dim=-1) # [batch, 1, e_dim]

```

```

scores_err = (e_curr_norm @ e_hist_norm.transpose(-1, -2)).squeeze(1) #
alpha_err = F.softmax(scores_err, dim=-1) # [batch, T]

# --- 4. Learned gating ---
gate_input = torch.cat([h_t, e_curr, x_p], dim=-1) # [batch, gate_input_]
g = F.softmax(self.W_gate(gate_input), dim=-1) # [batch, 3]
g1, g2, g3 = g[:, 0:1], g[:, 1:2], g[:, 2:3] # [batch, 1] each

# --- 5. Combined attention ---
alpha = g1 * alpha_time + g2 * alpha_phys + g3 * alpha_err # [batch, T]

# --- 6. Context vector ---
context = (alpha.unsqueeze(-1) * H_enc).sum(dim=1) # [batch, d_k]

return context, alpha

def attention_regularization_loss(self, alpha: Tensor, lambda_sparse: float =
    """
    Regularize attention weights: balance entropy (avoid collapse) with spars
    
$$L_{\text{attn}} = -\sum \alpha \log \alpha + \lambda_{\text{sparse}} \|\alpha\|_1$$

    """
    eps = 1e-8
    entropy_term = -(alpha * (alpha + eps).log()).sum(dim=-1).mean()
    sparsity_term = alpha.abs().sum(dim=-1).mean()
    return -entropy_term + lambda_sparse * sparsity_term

```

5.2 Port-Hamiltonian Physics Decoder

Create `src/pits_mras/models/decoders.py`:

```

"""
Port-Hamiltonian physics decoder for the PITNN.

Enforces energy conservation and positive dissipation by separating dynamics into
conservative (Hamiltonian) and dissipative components, plus a temporal correction

All mathematical identities from §3.1 are enforced here.
"""
import torch
import torch.nn as nn
from torch import Tensor
import torch.nn.functional as F
from pits_mras.utils.hamiltonian import (
    make_skew_symmetric, make_positive_definite,
    port_hamiltonian_energy_loss, hamiltonian_positivity_loss,
)

```

```

class HamiltonianNet(nn.Module):
    """
    Learns the Hamiltonian  $H_\theta(q, p)$  – a scalar positive-definite function
    representing total energy. Architecture: MLP with softplus output to ensure
    """
    def __init__(self, n_q: int, hidden_dim: int = 64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2 * n_q, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, 1),
            nn.Softplus(), # guarantees  $H > 0$ 
        )

    def forward(self, q: Tensor, p: Tensor) -> Tensor:
        """Returns  $H_\theta(q, p)$ , shape [batch, 1]."""
        return self.net(torch.cat([q, p], dim=-1))

```

```

class DissipationNet(nn.Module):
    """
    Learns the Cholesky factor  $L_\theta(q)$  of the dissipation matrix  $R_\theta = L^T L \geq 0$ .
    Outputs a lower-triangular matrix to ensure positive semi-definiteness.
    """
    def __init__(self, n_q: int, hidden_dim: int = 32):
        super().__init__()
        self.n_q = n_q
        self.net = nn.Sequential(
            nn.Linear(n_q, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, n_q * n_q), # will be reshaped to [n_q, n_q]
        )

    def forward(self, q: Tensor) -> Tensor:
        """Returns  $R_\theta(q) = L^T L$  of shape [batch, n_q, n_q]."""
        batch = q.shape[0]
        L_flat = self.net(q) # [batch, n_q * n_q]
        L = L_flat.view(batch, self.n_q, self.n_q)
        # Take lower-triangular part (diagonal must be positive for proper Choles
        L = torch.tril(L)
        L = L + torch.diag_embed(F.softplus(torch.diagonal(L, dim1=-2, dim2=-1)))
        return make_positive_definite(L) # [batch, n_q, n_q]

```

```

class PortHamiltonianDecoder(nn.Module):
    """
    Full port-Hamiltonian decoder implementing §3.1.

    Given context vector  $c_t$  and current state  $x_p = [q; p]$ :
    
$$\hat{f}_\theta = J(q)\nabla H_\theta - R_\theta(q)\dot{q} + B(x_p)u + W_{\text{corr}} c_t + b_{\text{corr}}$$

    """
    def __init__(
        self,
        n_q: int,                # generalized coordinate dimension
        context_dim: int,        # dimension of attention context  $c_t$ 
        output_dim: int,         # full output dimension (should be  $2*n_q$  for  $[q,p]$ 
        hamiltonian_hidden: int = 64,
        dissipation_hidden: int = 32,
        use_position_dependent_J: bool = False,
    ):
        super().__init__()
        self.n_q = n_q
        self.use_position_dependent_J = use_position_dependent_J

        self.H_net = HamiltonianNet(n_q, hamiltonian_hidden)
        self.L_net = DissipationNet(n_q, dissipation_hidden)

        if use_position_dependent_J:
            #  $J(q)$  for nonholonomic constraints
            self.J_net = nn.Sequential(
                nn.Linear(n_q, 32), nn.Tanh(),
                nn.Linear(32, (2 * n_q) ** 2)
            )
        else:
            # Constant canonical  $J$ :  $\begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix}$ 
            J_np = torch.zeros(2 * n_q, 2 * n_q)
            J_np[:n_q, n_q:] = torch.eye(n_q)
            J_np[n_q:, :n_q] = -torch.eye(n_q)
            self.register_buffer("J_canonical", J_np)

        # Input matrix  $B$  (state-dependent, small MLP)
        self.B_net = nn.Sequential(
            nn.Linear(2 * n_q, 32), nn.Tanh(),
            nn.Linear(32, 2 * n_q), # maps  $u$  (scalar  $\rightarrow$  vector; reshape as needed)
        )

        # Temporal correction from context
        self.W_corr = nn.Linear(context_dim, output_dim)

    def forward(

```

```

self,
q: Tensor,          # [batch, n_q]  generalized positions
p: Tensor,          # [batch, n_q]  generalized momenta
q_dot: Tensor,      # [batch, n_q]  velocity (for dissipation)
u: Tensor,          # [batch, control_dim]  control input
c_t: Tensor,        # [batch, context_dim]  attention context
) -> tuple[Tensor, Tensor, Tensor, Tensor]:
    """
    Returns (f_hat, H_val, P_diss, energy_loss).

    f_hat:          [batch, 2*n_q]  - full dynamics prediction
    H_val:          [batch, 1]      - Hamiltonian energy (for monitoring)
    P_diss:         [batch]         - dissipated power  $\nabla H^T R \nabla H$ 
    energy_loss: scalar             - port-Hamiltonian energy residual loss
    """
    batch = q.shape[0]
    qp = torch.cat([q, p], dim=-1) # [batch, 2*n_q]

    # 1. Hamiltonian and its gradient
    H_val = self.H_net(q, p) # [batch, 1]
    grad_H = torch.autograd.grad(
        H_val.sum(), qp, create_graph=True
    )[0] # [batch, 2*n_q]

    # 2. Interconnection matrix J
    if self.use_position_dependent_J:
        J_flat = self.J_net(q)
        J = J_flat.view(batch, 2 * self.n_q, 2 * self.n_q)
        J = make_skew_symmetric(J)
    else:
        J = self.J_canonical.unsqueeze(0).expand(batch, -1, -1) # [batch, 2n

    # 3. Conservative dynamics: f_cons = J ∇H
    f_cons = (J @ grad_H.unsqueeze(-1)).squeeze(-1) # [batch, 2*n_q]

    # 4. Dissipative dynamics: f_diss = -R q̇ (acts on velocity part only)
    R_theta = self.L_net(q) # [batch, n_q, n_q]
    f_diss_q = -(R_theta @ q_dot.unsqueeze(-1)).squeeze(-1) # [batch, n_q]
    f_diss = torch.cat([f_diss_q, torch.zeros_like(p)], dim=-1) # [batch, 2*

    # 5. Control input: f_ctrl = B(x_p) u
    B_val = self.B_net(qp) # [batch, 2*n_q]
    f_ctrl = B_val * u.sum(dim=-1, keepdim=True) # simplified; generalize

    # 6. Temporal correction from attention context
    f_corr = self.W_corr(c_t) # [batch, 2*n_q]

```

```

# 7. Total dynamics
f_hat = f_cons + f_diss + f_ctrl + f_corr          # [batch, 2*n_q]

# 8. Compute energy loss terms for physics loss L_physics
P_control = (B_val * u.sum(dim=-1, keepdim=True) * grad_H).sum(dim=-1)
grad_H_q = grad_H[:, :self.n_q]                  # [batch, n_q]
P_diss = (grad_H_q.unsqueeze(1) @ R_theta @ grad_H_q.unsqueeze(-1)).squeeze()
# dH/dt ≈ (f_cons + f_diss + f_ctrl) · ∇H (chain rule)
dH_dt = (f_hat * grad_H).sum(dim=-1)
energy_loss = port_hamiltonian_energy_loss(H_val.squeeze(), dH_dt, P_control)
energy_loss = energy_loss + hamiltonian_positivity_loss(H_val.squeeze())

return f_hat, H_val, P_diss, energy_loss

```

5.3 Critic / Value Network (NEW — Identity 1 & 2)

Create `src/pits_mras/models/critic.py`:

```

"""

```

Quadratic critic network implementing $\hat{V}(e) = W_c^T \phi_c(e)$.

This is the formal implementation of the ADP/integral-RL connection:

$\hat{V}(e) = e^T \hat{P} e$ where $\hat{P}$ is learned via the IRL Bellman error.

The quadratic basis guarantees:

- The LQR limit ($\hat{P} \rightarrow P_{\text{CARE}}$) is exactly representable.
- The critic gradient $\nabla_e \hat{V} = 2\hat{P}e$ is analytic.
- The costate identity $\lambda = \nabla \hat{V}$ is enforced by construction.

A small MLP residual can be added for the nonlinear extension.

```

"""

```

```

import torch
import torch.nn as nn
from torch import Tensor
from pits_mras.utils.lyapunov import quadratic_basis

```

```

class QuadraticCritic(nn.Module):

```

```

    """

```

Linear-in-parameters quadratic value function approximator.

$\hat{V}(e) = W^T \phi(e)$ where $\phi(e)$ is the upper-triangular Kronecker basis.

This parameterization ensures $\hat{V}$ is always a quadratic form $e^T \hat{P} e$

with $\hat{P} = \text{vech}^{-1}(W)$ (symmetric by construction of the basis).

If `nonlinear_residual=True`, adds a small MLP $\delta V(e)$ for the nonlinear regime.

```

"""

def __init__(
    self,
    state_dim: int,
    nonlinear_residual: bool = False,
    residual_hidden: int = 32,
):
    super().__init__()
    self.state_dim = state_dim
    self.basis_dim = state_dim * (state_dim + 1) // 2

    # Linear-in-parameters layer (no bias -  $V(0) = 0$  is required for a CLF)
    self.W_c = nn.Linear(self.basis_dim, 1, bias=False)

    # Ensure W_c initializes near the identity ( $\hat{P} \approx I$  at init)
    with torch.no_grad():
        # The diagonal elements of the basis correspond to  $e_i^2$ 
        # Initialize those to 1.0, off-diagonals to 0
        diag_indices = [i * (state_dim - i // 2) + i % (state_dim + 1)
                        for i in range(state_dim)]
        self.W_c.weight.data.zero_()
        for idx in range(state_dim):
            # Position of  $e_i^2$  in the upper-triangular basis
            pos = idx * state_dim - idx * (idx - 1) // 2
            self.W_c.weight.data[0, pos] = 1.0

    self.use_residual = nonlinear_residual
    if nonlinear_residual:
        self.residual_net = nn.Sequential(
            nn.Linear(state_dim, residual_hidden),
            nn.Tanh(),
            nn.Linear(residual_hidden, residual_hidden),
            nn.Tanh(),
            nn.Linear(residual_hidden, 1),
        )

def forward(self, e: Tensor) -> Tensor:
    """
    Returns  $\hat{V}(e)$  of shape [batch].
    Requires e to have grad enabled for the costate computation.
    """
    phi = quadratic_basis(e) # [batch, basis_dim]
    V = self.W_c(phi).squeeze(-1) # [batch]
    if self.use_residual:
        V = V + self.residual_net(e).squeeze(-1)
    return V

```

```

def gradient(self, e: Tensor) -> Tensor:
    """
    Compute  $\nabla_e \hat{V}$  via autograd. This IS the costate  $\hat{\lambda} = \nabla \hat{V}$ .
    Returns shape [batch, state_dim].
    """
    e = e.requires_grad_(True)
    V = self.forward(e)
    grad = torch.autograd.grad(
        V.sum(), e, create_graph=True
    )[0]
    return grad # [batch, state_dim]

def extract_P(self) -> Tensor:
    """
    Reconstruct the full symmetric  $\hat{P}$  matrix from the  $W_c$  weights.
    Returns shape [state_dim, state_dim].
    Useful for monitoring convergence of IRL to the CARE solution.
    """
    n = self.state_dim
    P = torch.zeros(n, n, device=self.W_c.weight.device,
                    dtype=self.W_c.weight.dtype)
    w = self.W_c.weight.data.squeeze(0) # [basis_dim]
    idx = 0
    for i in range(n):
        for j in range(i, n):
            if i == j:
                P[i, j] = w[idx]
            else:
                P[i, j] = w[idx] / 2.0
                P[j, i] = w[idx] / 2.0
            idx += 1
    return P

def positivity_loss(self) -> Tensor:
    """
    Penalize if  $\hat{P}$  is not positive definite.
    Loss = sum(ReLU(-min_eigenvalue)).
    """
    P = self.extract_P()
    eigvals = torch.linalg.eigvalsh(P)
    return torch.relu(-eigvals.min())

class CostateHead(nn.Module):
    """
    Implements the costate  $\hat{\lambda}(t) = \partial \hat{V} / \partial e$  and the optimal control  $u^* = -R^{-1}B^T \hat{\lambda}$ .

```

This enforces Identity 2 (PMP costate = critic gradient) by construction:
the action head is never an independent network – it IS the gradient of the c
"""

```
def __init__(self, critic: QuadraticCritic, R_inv: Tensor, B: Tensor):
    super().__init__()
    self.critic = critic
    self.register_buffer("R_inv", R_inv) # [control_dim, control_dim]
    self.register_buffer("B_mat", B) # [state_dim, control_dim]

def forward(self, e: Tensor) -> tuple[Tensor, Tensor]:
    """
    Returns (lambda_hat, u_optimal):
        lambda_hat: [batch, state_dim]
        u_optimal: [batch, control_dim]     $u^* = -R^{-1}B^T \nabla \hat{V}$ 
    """
    lambda_hat = self.critic.gradient(e) # [batch, state_dim]
    u_opt = -(lambda_hat @ self.B_mat) @ self.R_inv.T # [batch, control_dim]
    return lambda_hat, u_opt
```

5.4 Main PITNN

Create `src/pits_mras/models/pitnn.py`:

```
"""
Physics-Informed Temporal Neural Network (PITNN).

Implements Algorithm 1 from the PITS-MRAS technical specification.
The complete forward pass:
1. Input normalization and embedding
2. Causal LSTM encoder (forward-only, no information leakage from future)
3. Multi-head physics-informed attention (temporal + physical + error-driven)
4. Port-Hamiltonian physics decoder
```

Mathematical guarantees:

- Causality: forward-only LSTM, no future data in inference
- Energy conservation: port-Hamiltonian structure
- Physical plausibility: positive dissipation ($R_\theta = L^T L \geq 0$)

```
"""
import torch
import torch.nn as nn
from torch import Tensor
from pits_mras.models.attention import PhysicsInformedAttention
from pits_mras.models.decoders import PortHamiltonianDecoder
from pits_mras.config import NetworkConfig, PhysicsConfig
```

```

class PITNN(nn.Module):
    """
    Physics-Informed Temporal Neural Network – the core dynamics model.

    Takes a sliding window of (state, control) history and the current tracking
    error, and outputs a dynamics prediction  $\hat{f}_\theta(x_p, u, t)$  for the plant.
    """

    def __init__(self, net_cfg: NetworkConfig, phys_cfg: PhysicsConfig):
        super().__init__()
        self.input_dim = net_cfg.input_dim
        self.hidden_dim = net_cfg.hidden_dim
        self.output_dim = net_cfg.output_dim
        self.n_q = phys_cfg.n_generalized_coords

        # — Input normalization (running statistics, non-trainable) —
        self.register_buffer("mu_x", torch.zeros(net_cfg.input_dim))
        self.register_buffer("sigma_x", torch.ones(net_cfg.input_dim))

        # — Embedding —
        self.embed_state = nn.Linear(net_cfg.input_dim, net_cfg.embedding_dim)
        self.embed_control = nn.Linear(net_cfg.input_dim, net_cfg.embedding_dim)

        # — Causal LSTM encoder —
        lstm_input = net_cfg.embedding_dim * 2  # state + control embeddings
        self.lstm = nn.LSTM(
            input_size=lstm_input,
            hidden_size=net_cfg.hidden_dim,
            num_layers=net_cfg.lstm_layers,
            batch_first=True,
        )

        # — Physics-informed attention —
        self.attention = PhysicsInformedAttention(
            d_k=net_cfg.hidden_dim,
            e_dim=net_cfg.output_dim,  # error dim  $\approx$  output dim for this system
            n_state=net_cfg.input_dim,
            n_heads=net_cfg.attention_heads,
        )

        # — Port-Hamiltonian decoder —
        self.decoder = PortHamiltonianDecoder(
            n_q=phys_cfg.n_generalized_coords,
            context_dim=net_cfg.hidden_dim,
            output_dim=net_cfg.output_dim,

```

```

        hamiltonian_hidden=phys_cfg.hamiltonian_hidden,
        dissipation_hidden=phys_cfg.dissipation_hidden,
        use_position_dependent_J=phys_cfg.use_position_dependent_J,
    )

def update_normalization(self, x_data: Tensor) -> None:
    """Update running normalization statistics from a data batch."""
    self.mu_x.copy_(x_data.mean(dim=0))
    self.sigma_x.copy_(x_data.std(dim=0).clamp(min=1e-6))

def normalize(self, x: Tensor) -> Tensor:
    return (x - self.mu_x) / self.sigma_x

def forward(
    self,
    x_hist: Tensor,          # [batch, T, input_dim]  plant state history
    u_hist: Tensor,          # [batch, T, input_dim]  control history
    x_p_curr: Tensor,        # [batch, input_dim]    current plant state
    u_curr: Tensor,          # [batch, control_dim]  current control
    e_curr: Tensor,          # [batch, e_dim]       current tracking error
    e_hist: Tensor,          # [batch, T, e_dim]    error history
) -> dict:
    """
    Forward pass (Algorithm 1).
    Returns a dict with keys: 'f_hat', 'context', 'alpha', 'H_val',
                                'P_diss', 'energy_loss', 'attn_reg_loss'
    """
    batch, T, _ = x_hist.shape

    # 1. Normalize and embed
    x_norm = self.normalize(x_hist)                # [batch, T, input_dim]
    e_state = self.embed_state(x_norm)              # [batch, T, emb_dim]
    e_ctrl = self.embed_control(self.normalize(u_hist)) # [batch, T, emb_dim]
    seq = torch.cat([e_state, e_ctrl], dim=-1)      # [batch, T, 2*emb_dim]

    # 2. Causal LSTM (no bidirectional – preserves causality for deployment)
    H_enc, _ = self.lstm(seq) # [batch, T, hidden_dim]

    # 3. Compute velocity approximation (finite difference) for dissipation
    if T > 1:
        x_p_dot = (x_hist[:, -1, :] - x_hist[:, -2, :]) / 0.01 # [batch, inp
    else:
        x_p_dot = torch.zeros_like(x_p_curr)

    # 4. Physics-informed attention
    context, alpha = self.attention(
        H_enc, e_hist, x_p_curr, e_curr, x_p_dot, u_curr

```

```

) # context: [batch, hidden_dim], alpha: [batch, T]

# 5. Port-Hamiltonian decoder – extract [q, p] from state
q = x_p_curr[:, :self.n_q] # [batch, n_q]
p = x_p_curr[:, self.n_q:2*self.n_q] # [batch, n_q]
q_dot = x_p_dot[:, :self.n_q]

f_hat, H_val, P_diss, energy_loss = self.decoder(q, p, q_dot, u_curr, con

# 6. Attention regularization
attn_reg = self.attention.attention_regularization_loss(alpha)

return {
    "f_hat": f_hat, # [batch, output_dim] dynamics predictio
    "context": context, # [batch, hidden_dim] attention context
    "alpha": alpha, # [batch, T] attention weights
    "H_val": H_val, # [batch, 1] Hamiltonian energy
    "P_diss": P_diss, # [batch] dissipated power
    "energy_loss": energy_loss, # scalar energy conservati
    "attn_reg_loss": attn_reg, # scalar attention regulari
}

```

§6 · Phase 3 — Loss Functions

6.1 Physics Loss

Create `src/pits_mras/losses/physics.py` implementing the four components of L_{physics} from §2.2 of the technical spec: $L_{\text{physics}} = \lambda_1 L_{\text{energy}} + \lambda_2 L_{\text{PDE}} + \lambda_3 L_{\text{BC}} + \lambda_4 L_{\text{sym}}$.

The energy loss is already computed by `PortHamiltonianDecoder.forward()`. This module handles the PDE residual (system-specific, passed as a callable), boundary conditions, and symmetry. Provide a `PhysicsLoss` class with `__init__(pde_operator, bc_points, sym_transform, weights)` and `forward(f_hat, x, u, t, energy_loss)`.

6.2 Temporal Loss

Create `src/pits_mras/losses/temporal.py` implementing:

- `MultiStepPredictionLoss` : $L_{\text{pred}} = \sum_{k=1}^K w_k \|x(t+k\Delta t) - \hat{x}^k(k)\|^2$
- `AttentionRegularizationLoss` : wrapper around the PITNN's `attn_reg_loss`
- `TemporalSmoothnessLoss` : $L_{\text{smooth}} = \|\partial \hat{f} / \partial t\|^2$ (finite difference approximation)

- `TemporalLoss(total)` : combines all three with α_1, α_2 weights.

6.3 MRAS Stability Loss

Create `src/pits_mras/losses/stability.py` implementing:

- `LyapunovConstraintLoss` : $L_{\text{Lyap}} = E[\max(0, \dot{V} + \mu V)^2]$
- `ParameterBoundednessLoss` : $L_{\text{param}} = \|\theta\|^2 + \|\theta_c\|^2$
- `ControlEffortLoss` : $L_{\text{ctrl}} = E[\|u\|^2 + \lambda_{\Delta u} \|\Delta u\|^2]$
- `MRASStabilityLoss(total)` : combines with β_1, β_2 weights.

Import `lyapunov_derivative` from `utils/lyapunov.py` for computing $\dot{V}$.

6.4 Integral RL Loss (NEW — Identity 1)

Create `src/pits_mras/losses/irl.py`:

```
"""
Integral Reinforcement Learning (IRL) Bellman loss.

Implements the model-free policy evaluation equation (Vrabie & Lewis 2009):

$$V(x(t)) = \int_{\{t\}^{\{t+T\}}} r(x, u) \, d\tau + V(x(t+T))$$


Bellman error:  $\delta_{\text{IRL}}(t) = \int_{\{t-T\}^{\{t\}}} r \, d\tau - [\hat{V}(e(t)) - \hat{V}(e(t-T))]$ 

Key property: this equation does NOT contain the drift dynamics  $A_m$ ,
making policy evaluation model-free from measured state/control trajectories.

Reference: Vrabie & Lewis (2009), Neural Networks 22(3):237-246.
DOI: 10.1016/j.neunet.2009.03.008
"""
import torch
import torch.nn as nn
from torch import Tensor
from collections import deque

class IRLBellmanAccumulator:
    """
    Maintains a sliding window buffer for computing IRL integrals numerically.
    Uses the trapezoidal rule for  $\int r \, d\tau$ .
    """

    def __init__(self, window_size: int, device: str = "cpu"):
```

```

        self.window_size = window_size
        self.device = device
        self._e_buffer: deque = deque(maxlen=window_size + 1)
        self._r_buffer: deque = deque(maxlen=window_size + 1)

def push(self, e: Tensor, r: Tensor) -> None:
    """Add (e, r) pair where  $r = e^T Q e + u^T R u$  is the instantaneous cost."""
    self._e_buffer.append(e.detach())
    self._r_buffer.append(r.detach())

def is_ready(self) -> bool:
    return len(self._e_buffer) >= self.window_size + 1

def compute_integral_r(self, dt: float) -> Tensor:
    """Trapezoidal integration of r over the window. Returns scalar."""
    r_tensor = torch.stack(list(self._r_buffer)) # [window+1, batch]
    # Trapezoidal:  $(r[0]/2 + r[1] + \dots + r[n-1] + r[n]/2) * dt$ 
    integral = (r_tensor[0] / 2 + r_tensor[1:-1].sum(0) + r_tensor[-1] / 2) * dt
    return integral # [batch]

def get_e_endpoints(self) -> tuple[Tensor, Tensor]:
    """Return  $e(t-T)$  and  $e(t)$  for the  $\Delta V$  computation."""
    return self._e_buffer[0], self._e_buffer[-1]

class IRLBellmanLoss(nn.Module):
    """
    Integral RL Bellman error loss for critic training.

    With critic  $\hat{V}(e) = W^T \phi(e)$ , the loss is:
    
$$L_{\text{IRL}} = \frac{1}{2} E[ (\int r dt - \Delta \hat{V})^2 ]$$


    Minimizing this over  $W_c$  makes  $\hat{V}$  converge to the true value function  $V^*$ ,
    which equals  $e^T P^* e$  where  $P^*$  solves the CARE for the tracking-error system.
    This is Kleinman's policy iteration run online without knowing  $A$ .
    """

def __init__(self, Q: Tensor, R: Tensor, window_size: int, dt: float):
    super().__init__()
    self.register_buffer("Q", Q)
    self.register_buffer("R", R)
    self.window_size = window_size
    self.dt = dt

def running_cost(self, e: Tensor, u: Tensor) -> Tensor:
    """ $r(e, u) = e^T Q e + u^T R u$ . Shapes: e [batch, n], u [batch, m]."""
    r_e = (e @ self.Q * e).sum(dim=-1) # [batch]

```

```

        r_u = (u @ self.R * u).sum(dim=-1) # [batch]
        return r_e + r_u # [batch]

def forward(
    self,
    critic: nn.Module,
    accumulator: IRLBellmanAccumulator,
) -> Tensor:
    """
    Compute the IRL Bellman loss from the accumulator buffer.
    Returns scalar loss or zero if buffer not ready.
    """
    if not accumulator.is_ready():
        return torch.tensor(0.0)

    integral_r = accumulator.compute_integral_r(self.dt) # [batch]
    e_start, e_end = accumulator.get_e_endpoints()
    delta_V = critic(e_end) - critic(e_start) # [batch]
    delta_irl = integral_r - delta_V # [batch]
    return 0.5 * (delta_irl ** 2).mean()

```

6.5 HJB Residual Loss (NEW — Identity 8)

Create `src/pits_mras/losses/hjb.py` implementing §3.5:

- Class `HJBResidualLoss(Q, R, R_inv, A_m, B)`
- `forward(e, f_corr, critic, costate_head)` computes the HJB PDE residual and returns $\| e^T Q e + (u^*)^T R u^* + \nabla \hat{V} \cdot (A_m e + B u^* + f_{\text{corr}}) \|^2$.
- Include a `LyapunovDecreaseEnforcer` loss: $L_{\text{dec}} = E[\text{ReLU}(\nabla \hat{V} \cdot \hat{f} + \ell)]$ that directly penalizes $\dot{V} > -\ell$ (tighter than the existing `L_Lyap`).

6.6 Total Loss Aggregator

Create `src/pits_mras/losses/__init__.py` with a `TotalLoss` class that wraps all sub-losses and applies the weights from `LossConfig`. It should log each sub-loss separately to TensorBoard/wandb under the names: `loss/physics`, `loss/temporal`, `loss/stability`, `loss/irl`, `loss/hjb`, `loss/costate`, `loss/data`.

§7 • Phase 4 — Controllers

7.1 Reference Models

Create src/pits_mras/controllers/reference_models.py:

```
"""
Reference model implementations for PITS-MRAS.

The reference model defines the desired closed-loop behavior:
 $\dot{x}_m = A_m x_m + B_m r, \quad y_m = C_m x_m$ 

 $A_m$  must be Hurwitz (all eigenvalues strictly negative real parts).
This is verified at construction time.

Mathematical note:  $A_m$  defines the target tracking dynamics; the MRAS Lyapunov
equation  $A_m^T P + P A_m = -Q$  uses this same  $A_m$  to compute  $P$ , linking the
reference model directly to the value function (Identity 1).
"""

import torch
import torch.nn as nn
from torch import Tensor
import numpy as np
from pits_mras.utils.lyapunov import solve_lyapunov, check_hurwitz, kleinman_iter

class LinearReferenceModel(nn.Module):
    """Standard linear MRAS reference model."""

    def __init__(
        self,
        A_m: np.ndarray,
        B_m: np.ndarray,
        C_m: np.ndarray,
        Q: np.ndarray,
        R: np.ndarray,
    ):
        super().__init__()
        if not check_hurwitz(A_m):
            raise ValueError("A_m must be Hurwitz (all eigenvalues < 0 real parts)")

        n = A_m.shape[0]
        m = B_m.shape[1]

        # Solve Lyapunov equation for P (policy evaluation step, Identity 1)
        P = solve_lyapunov(A_m, Q)
        # Run Kleinman iteration to get the LQR gain K_opt
        P_opt, K_opt = kleinman_iteration(A_m, B_m, Q, R)

        self.register_buffer("A_m", torch.tensor(A_m, dtype=torch.float32))
```

```

self.register_buffer("B_m", torch.tensor(B_m, dtype=torch.float32))
self.register_buffer("C_m", torch.tensor(C_m, dtype=torch.float32))
self.register_buffer("Q", torch.tensor(Q, dtype=torch.float32))
self.register_buffer("R", torch.tensor(R, dtype=torch.float32))
self.register_buffer("R_inv", torch.tensor(np.linalg.inv(R), dtype=torch.
self.register_buffer("P", torch.tensor(P, dtype=torch.float32))
self.register_buffer("P_opt", torch.tensor(P_opt, dtype=torch.float32))
self.register_buffer("K_opt", torch.tensor(K_opt, dtype=torch.float32))

def step(self, x_m: Tensor, r: Tensor, dt: float) -> Tensor:
    """Euler integration of  $\dot{x}_m = A_m x_m + B_m r$ ."""
    dx_m = x_m @ self.A_m.T + r @ self.B_m.T
    return x_m + dx_m * dt

def output(self, x_m: Tensor) -> Tensor:
    return x_m @ self.C_m.T

```

7.2 CLF-CBF Safety Filter (NEW — Identity 3)

Create `src/pits_mras/controllers/safety.py`:

```

"""
CLF-CBF-QP safety filter for the MRAS tracking-error system.

Implements the closed-form single-constraint CBF projection from §3.4:

$$h(e) = c - e^T P e \quad (\text{safe} = \text{tracking error inside the P-ellipsoid})$$


$$u_{\text{safe}} = u_{\text{nom}} - [(L_f h + L_g h \cdot u_{\text{nom}} + \gamma h)_+ / \|L_g h\|^2] \cdot (L_g h)^T$$

where  $(\cdot)_+ = \max(0, \cdot)$  is the ReLU projection (no correction if safe).

The Lie derivatives along the reference model and control directions are:

$$L_f h(e) = -2 e^T P A_m e \quad (A_m \text{ is the reference model matrix})$$


$$L_g h(e) = -2 e^T P B \quad (B \text{ is the control input matrix})$$


Mathematical guarantee: forward invariance of  $\{e : h(e) \geq 0\} = \{e : e^T P e \leq c\}$ .
Reference: Ames et al. (2017), IEEE TAC. arXiv:1609.06408.
"""

import torch
import torch.nn as nn
from torch import Tensor
import torch.nn.functional as F

class CLFCBFSafetyFilter(nn.Module):
    """
    Rigorous CBF-based safety filter replacing the heuristic  $\dot{V} < 0$  check.

```

The same P matrix used for the Lyapunov CLF ($V=e^TPe$) also serves as the CBF ($h=c-e^TPe$), meaning one P matrix simultaneously certifies stability (CLF: $\dot{V} < 0$) and safety (CBF: forward invariance of error ellipsoid).

"""

```
def __init__(
    self,
    P: Tensor,          # [n, n] Lyapunov matrix from reference model
    A_m: Tensor,        # [n, n] reference model dynamics
    B_ctrl: Tensor,     # [n, m] control input matrix
    safety_margin: float = 10.0,  # c: ellipsoid size  $h(e) = c - e^TPe$ 
    decay_rate: float = 1.0,     #  $\gamma$ : class-K function rate
):
    super().__init__()
    self.register_buffer("P", P)
    self.register_buffer("A_m", A_m)
    self.register_buffer("B_ctrl", B_ctrl)
    self.safety_margin = safety_margin
    self.decay_rate = decay_rate
    self._n_corrections = 0      # diagnostic counter

def forward(self, e: Tensor, u_nom: Tensor) -> tuple[Tensor, Tensor, Tensor]:
    """
    Apply the CBF safety filter.

    Args:
        e:      [batch, n] tracking error
        u_nom:  [batch, m] nominal control (from MRAS controller or RL polic

    Returns:
        u_safe:  [batch, m] safety-filtered control
        h_val:   [batch] CBF value  $h(e) = c - e^TPe$  (>0 = safe)
        slack:   [batch] correction magnitude (0 = filter inactive)
    """
    # CBF value
    ePe = (e @ self.P * e).sum(dim=-1) # [batch]  $e^TPe$ 
    h_e = self.safety_margin - ePe      # [batch]  $h(e) = c - e^TPe$ 

    # Lie derivatives
    #  $L_f h = dh/de \cdot f(e) = -2e^TP \cdot A_m e$ 
    Pe = e @ self.P # [batch, n]
    L_f_h = -2.0 * (Pe * (e @ self.A_m.T)).sum(dim=-1) # [batch]

    #  $L_g h = dh/de \cdot g = -2e^TP \cdot B$ 
    L_g_h = -2.0 * (Pe @ self.B_ctrl) # [batch, m]
```

```

# CBF constraint:  $L_f h + (L_g h) \cdot u \geq -\gamma h(e)$ 
# Safety index  $a = L_f h + (L_g h) \cdot u_{\text{nom}} + \gamma h(e)$ 
# Negative a means the nominal control violates the constraint.
L_g_h_u_nom = (L_g_h * u_nom).sum(dim=-1) # [batch]
a = L_f_h + L_g_h_u_nom + self.decay_rate * h_e # [batch]

# Closed-form minimum-norm correction (only applied where  $a < 0$ )
L_g_h_sq = (L_g_h * L_g_h).sum(dim=-1) + 1e-8 # [batch], avoid div/0
correction_scale = F.relu(-a) / L_g_h_sq # [batch], = 0 when safe
correction = correction_scale.unsqueeze(-1) * L_g_h # [batch, m]
u_safe = u_nom + correction # [batch, m]
slack = correction.norm(dim=-1) # [batch]

return u_safe, h_e, slack

def cbf_constraint_loss(self, e: Tensor, u: Tensor) -> Tensor:
    """
    Soft CBF constraint loss for training (penalize violations of  $h(e) \geq 0$ 
    and the forward-invariance condition). Can be added to  $L_{\text{total}}$ .
    """
    _, h_e, _ = self.forward(e, u)
    # Penalize safety constraint violations ( $h < 0$ ) and decay violations
    return F.relu(-h_e).mean()

```

7.3 MRAS Controller (Updated with Actor-Critic)

Create `src/pits_mras/controllers/mras.py`:

```

"""
MRAS Adaptive Controller with actor-critic upgrade.

Classical MRAS structure:
     $u(t) = K_{fb}(t) e(t) + K_{ff}(t) r(t) + u_{aux}(t)$ 

New actor-critic upgrade (Identity 4 – DPG connection):
    The feedback gain is initialized from the LQR solution  $K_{opt} = R^{-1}B^T P_{opt}$ 
    and then improved via the IRL Bellman update.

    Classical update:  $\dot{\theta}_c = -\Gamma_c [\nabla L + \gamma_{MRAS} e \phi_c]$ 
    Upgraded update:  $\dot{\theta}_c = -\Gamma_c [\nabla L + \phi_c \cdot \nabla_a \hat{Q}]$  (DPG-style)

The CBF safety filter wraps the nominal control output.
"""
import torch
import torch.nn as nn
from torch import Tensor

```

```

from pits_mras.models.critic import QuadraticCritic, CostateHead
from pits_mras.controllers.safety import CLFCBFSafetyFilter
from pits_mras.controllers.reference_models import LinearReferenceModel

class MRASController(nn.Module):
    """
    Full MRAS adaptive controller combining:
    - Classical MRAS feedback/feedforward structure
    - IRL critic-guided actor update (Identity 1, 4)
    - Costate-based optimal control action (Identity 2)
    - CLF-CBF safety filter (Identity 3)
    """

    def __init__(
        self,
        ref_model: LinearReferenceModel,
        state_dim: int,
        control_dim: int,
        hidden_dim: int = 64,
        use_cbf: bool = True,
        safety_margin: float = 10.0,
        nonlinear_critic: bool = False,
    ):
        super().__init__()
        self.state_dim = state_dim
        self.control_dim = control_dim

        # — Critic (IRL value function, Identity 1) —
        self.critic = QuadraticCritic(state_dim, nonlinear_residual=nonlinear_cri

        # — Costate head (optimal action =  $-R^{-1}B^T\hat{V}\hat{V}$ , Identity 2) —
        self.costate_head = CostateHead(
            critic=self.critic,
            R_inv=ref_model.R_inv,
            B=ref_model.B_m,
        )

        # — Feedforward gain (learns  $r \rightarrow u$  mapping) —
        self.K_ff = nn.Linear(state_dim, control_dim, bias=False)

        # — Auxiliary compensation (disturbance + uncertainty) —
        self.compensator = nn.Sequential(
            nn.Linear(ref_model.A_m.shape[0] + state_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, control_dim),
        )

```

```

# — CBF safety filter (Identity 3) —
self.use_cbf = use_cbf
if use_cbf:
    self.safety_filter = CLFCBFSafetyFilter(
        P=ref_model.P,
        A_m=ref_model.A_m,
        B_ctrl=ref_model.B_m,
        safety_margin=safety_margin,
    )

# — Initialize feedback gain from LQR solution (warm start) —
with torch.no_grad():
    self.critic.W_c.weight.data.zero_()
    # Initialize critic near the LQR value function:  $P \approx P_{opt}$ 
    P_opt = ref_model.P_opt
    n = state_dim
    idx = 0
    w = torch.zeros(n * (n + 1) // 2)
    for i in range(n):
        for j in range(i, n):
            if i == j:
                w[idx] = P_opt[i, j].item()
            else:
                w[idx] = 2 * P_opt[i, j].item() # off-diagonal appears o
            idx += 1
    self.critic.W_c.weight.data[0] = w

def forward(
    self,
    e: Tensor,      # [batch, state_dim] tracking error
    x_p: Tensor,    # [batch, state_dim] plant state
    x_m: Tensor,    # [batch, state_dim] reference model state
    r: Tensor,      # [batch, state_dim] reference command
    context: Tensor, # [batch, hidden_dim] PITNN attention context
) -> dict:
    """
    Compute control action.
    Returns dict with 'u_nom', 'u_safe', 'lambda_hat', 'V_hat', 'h_cbf', 'cbf'
    """
    # 1. Optimal feedback:  $u_{fb} = -R^{-1}B^T V \hat{V}$  (costate head, Identity 2)
    lambda_hat, u_fb = self.costate_head(e)

    # 2. Feedforward:  $u_{ff} = K_{ff} r$ 
    u_ff = self.K_ff(r)

    # 3. Auxiliary: compensates context-predicted disturbances

```

```

u_aux = self.compensator(torch.cat([context[:, :self.state_dim], x_p], di

# 4. Nominal control
u_nom = u_fb + u_ff + u_aux

# 5. CBF safety filter
if self.use_cbf:
    u_safe, h_cbf, cbf_slack = self.safety_filter(e, u_nom)
else:
    u_safe = u_nom
    h_cbf = torch.zeros(e.shape[0], device=e.device)
    cbf_slack = torch.zeros_like(h_cbf)

V_hat = self.critic(e)

return {
    "u_nom": u_nom,
    "u_safe": u_safe,
    "lambda_hat": lambda_hat,
    "V_hat": V_hat,
    "h_cbf": h_cbf,
    "cbf_slack": cbf_slack,
}

def mras_regressor(self, e: Tensor, r: Tensor, x_p: Tensor) -> Tensor:
    """
    Classical MRAS regressor:  $\phi_c = [e^T, r^T, x_p^T]^T$ .
    Used in both the classical and DPG-style adaptation laws.
    """
    return torch.cat([e, r, x_p], dim=-1) # [batch, 3*state_dim]

```

§8 · Phase 5 — Training Pipelines

8.1 Pre-Training (Algorithm 2)

Create `src/pits_mras/training/pretrain.py` implementing the three-stage curriculum:

Stage 1A (epochs 1–1000): Minimize only $\lambda_{\text{physics}} L_{\text{physics}} + 0.1 L_{\text{data}}$. Sample collocation points uniformly in the state-control-time domain.

Stage 1B (epochs 1001–3000): Cosine annealing of λ_{data} from 0.1 to 1.0. $\lambda_{\text{data}}(\text{epoch}) = 0.1 + 0.9 \cdot (1 - \cos(\pi(\text{epoch}-1000)/2000)) / 2$

Stage 1C (epochs 3001–5000): Activate LSTM/attention by adding L_{temporal} with linear

warm-up. $\lambda_{\text{temp}}(\text{epoch}) = \lambda_{\text{temp_final}} \cdot (\text{epoch} - 3000) / 2000$

Validation criterion: $L_{\text{physics}} < \varepsilon_{\text{tol}}$ must hold throughout. If it spikes above threshold, reduce the data weight λ_{data} by 0.5 and log a warning.

8.2 Co-Training with IRL Update (Algorithm 3, Extended)

Create `src/pits_mras/training/cotrain.py`. This is the most critical training file. Extend Algorithm 3 with the following additions after the standard gradient update loop (line 52 of the existing algorithm):

```
# — NEW: IRL critic update (Identity 1 – Vrabie & Lewis 2009) —
r_inst = irl_loss.running_cost(e, u_safe) # instantaneous cost r(e,u)
irl_accumulator.push(e.detach(), r_inst.detach())

if irl_accumulator.is_ready():
    L_irl = irl_loss(controller.critic, irl_accumulator)
    L_irl_total += L_irl

    # Critic gradient step (policy evaluation)
    critic_optimizer.zero_grad()
    L_irl.backward()
    torch.nn.utils.clip_grad_norm_(controller.critic.parameters(), max_norm=1.0)
    critic_optimizer.step()

    # Policy improvement step: update  $K_{\text{fb}} \leftarrow R^{-1}B^T \hat{P}$ 
    with torch.no_grad():
        P_hat = controller.critic.extract_P()
        K_new = ref_model.R_inv @ ref_model.B_m.T @ P_hat
        # Soft update of the effective feedback gain via costate head (no explici
        # The costate head already uses  $\nabla \hat{V}$ , which IS  $K_{\text{fb}} e$  when  $\hat{V} = e^T \hat{P} e$ 

# — NEW: HJB residual update —
if cfg.losses.lambda_hjb > 0:
    L_hjb = hjb_loss(e, pitnn_output["context"][:, :output_dim], controller.criti
    L_total += cfg.losses.lambda_hjb * L_hjb

# — NEW: Costate consistency loss —
lambda_hat = controller_output["lambda_hat"]
grad_V = controller.critic.gradient(e)
L_costate = (lambda_hat - grad_V).pow(2).mean()
L_total += cfg.losses.lambda_costate * L_costate

# — NEW: Critic positivity regularization —
L_pos = controller.critic.positivity_loss()
L_total += 1e-3 * L_pos
```

```
# — CBF constraint loss —
if cfg.safety.enable_cbf:
    L_cbf = controller.safety_filter.cbf_constraint_loss(e, u_safe)
    L_total += 0.1 * L_cbf
```

The co-training loop also needs two optimizer objects: `optimizer_pitnn` for PITNN parameters (Adam, $\text{lr}=1\text{e-}4$) and `critic_optimizer` for the critic (Adam, $\text{lr}=1\text{e-}3$, separate because the IRL update has a different learning rate).

8.3 IRL Standalone Trainer

Create `src/pits_mras/training/irl_trainer.py` for offline pre-training of the critic from a fixed dataset of trajectories. This is useful when you have existing demonstration data and want to initialize $\hat{P}$ close to P_{opt} before the co-training loop begins. The standalone trainer runs Kleinman-style batch least-squares on the IRL Bellman equations and stops when $\|\hat{P} - P_{\text{opt}}\|_F / \|P_{\text{opt}}\|_F < 0.01$.

§9 · Phase 6 — Inference Engine

9.1 Real-Time Inference with CBF

Create `src/pits_mras/inference/realtime.py`:

```
"""
Real-time inference engine for PITS-MRAS.

Implements the 4-step closed-loop inference:
1. Measure plant state  $x_p$ 
2. Update history buffer (bounded deque)
3. PITNN forward pass  $\rightarrow$  dynamics prediction  $\hat{f}_\theta$ 
4. Reference model step  $\rightarrow$  compute error  $e$ 
5. Controller forward pass  $\rightarrow u_{\text{nom}}$ 
6. CBF safety filter  $\rightarrow u_{\text{safe}}$  (replaces heuristic  $\dot{V} < 0$  check)
7. Apply  $u_{\text{safe}}$  to plant;  $\log \hat{V}$ ,  $h_{\text{CBF}}$ ,  $\|e\|$ 

Thread-safe buffer management for parallel deployment (1 kHz control thread).
"""
import torch
from torch import Tensor
from collections import deque
import threading
from pits_mras.models.pitnn import PITNN
```

```

from pits_mras.controllers.mras import MRASController
from pits_mras.controllers.reference_models import LinearReferenceModel

class RealtimeInferenceEngine:
    def __init__(
        self,
        pitnn: PITNN,
        controller: MRASController,
        ref_model: LinearReferenceModel,
        horizon: int = 50,
        device: str = "cpu",
    ):
        self.pitnn = pitnn.eval()
        self.controller = controller.eval()
        self.ref_model = ref_model
        self.horizon = horizon
        self.device = device
        self._x_hist: deque = deque(maxlen=horizon)
        self._u_hist: deque = deque(maxlen=horizon)
        self._e_hist: deque = deque(maxlen=horizon)
        self._x_m = None      # reference model state (initialized on first call)
        self._lock = threading.Lock()

    @torch.no_grad()
    def step(
        self,
        x_p: Tensor,      # [state_dim] - current plant state (no batch dim)
        r: Tensor,         # [state_dim] - current reference command
        dt: float = 0.01,
    ) -> dict:
        """
        Execute one control cycle.
        Returns dict with 'u_safe', 'e', 'V_hat', 'h_cbf', 'f_hat', 'cbf_active'.
        """
        with self._lock:
            if self._x_m is None:
                self._x_m = x_p.clone()
                # Initialize history buffers with current state
                for _ in range(self.horizon):
                    self._x_hist.append(x_p)
                    self._u_hist.append(torch.zeros(self.ref_model.B_m.shape[1],
                                                    device=self.device))
                    self._e_hist.append(torch.zeros_like(x_p))

            # Build batched history tensors (add batch dim)
            x_hist = torch.stack(list(self._x_hist)).unsqueeze(0)      # [1, T, n]

```

```

u_hist = torch.stack(list(self._u_hist)).unsqueeze(0)
e_hist = torch.stack(list(self._e_hist)).unsqueeze(0)

# Reference model step
self._x_m = self.ref_model.step(
    self._x_m.unsqueeze(0), r.unsqueeze(0), dt
).squeeze(0)
y_m = self.ref_model.output(self._x_m.unsqueeze(0)).squeeze(0)
y_p = x_p # simplified: C_p = I
e = y_p - y_m

# PITNN forward pass
u_prev = self._u_hist[-1]
pitnn_out = self.pitnn(
    x_hist, u_hist,
    x_p.unsqueeze(0), u_prev.unsqueeze(0),
    e.unsqueeze(0), e_hist,
)
f_hat = pitnn_out["f_hat"].squeeze(0)
context = pitnn_out["context"]

# Controller
ctrl_out = self.controller(
    e.unsqueeze(0), x_p.unsqueeze(0),
    self._x_m.unsqueeze(0), r.unsqueeze(0), context,
)
u_safe = ctrl_out["u_safe"].squeeze(0)
cbf_active = (ctrl_out["cbf_slack"] > 1e-4).any().item()

# Update history
self._x_hist.append(x_p)
self._u_hist.append(u_safe)
self._e_hist.append(e)

return {
    "u_safe": u_safe,
    "e": e,
    "V_hat": ctrl_out["V_hat"].squeeze(),
    "h_cbf": ctrl_out["h_cbf"].squeeze(),
    "f_hat": f_hat,
    "cbf_active": cbf_active,
}

```

9.2 Parallel Thread Architecture

Create `src/pits_mras/inference/parallel.py` implementing:

- `ControlThread` — 1 kHz thread calling `engine.step()` ; uses lock-free buffer
- `AdaptationThread` — 100 Hz thread that runs the IRL critic update and policy improvement from a shared replay buffer
- `MonitorThread` — 10 Hz thread computing $\hat{V}$, $\|e\|$, h_CBF , CBF activation rate

Use `threading.Event` for graceful shutdown. The adaptation thread must use a `copy.deepcopy` of the critic for the update and then atomically swap it back (double-buffer pattern to avoid mid-step reads of a partially-updated critic).

§10 • Phase 7 — Examples

10.1 Robotic Manipulator (`examples/robotic_manipulator.py`)

A 2-DOF planar manipulator: state $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]$ (generalized coordinates and velocities), control $u = [\tau_1, \tau_2]$ (joint torques). The Hamiltonian is the total mechanical energy $H = \frac{1}{2} \dot{q}^T M(q) \dot{q} + V(q)$. The reference model tracks a sinusoidal joint-angle trajectory. Show in a plot: (a) tracking error $\|e(t)\|$ over time, (b) $\hat{V}(e(t))$ Lyapunov value (should decrease monotonically after warm-up), (c) CBF activation flag (red markers when the filter corrects u), (d) $\|\hat{P} - P_CARE\|_F / \|P_CARE\|_F$ critic convergence metric.

10.2 Autonomous Vehicle (`examples/autonomous_vehicle.py`)

Lateral control at 80 km/h: state = [lateral error, heading error, yaw rate], control = [steering angle]. Reference model = first-order lateral dynamics. Include a wind-gust disturbance $\Delta(t) = 0.5 \sin(2\pi t/10)$ N·m to demonstrate robustness. Show the standard tracking plot plus a comparison: with CBF vs. without CBF, demonstrating that the CBF prevents lane-departure events.

10.3 Building HVAC (`examples/building_hvac.py`)

Thermal zone control: state = $[T_zone, T_wall, T_supply]$, control = [mass_flow]. The Hamiltonian is thermal energy $H = \sum (c_p m T^2/2)$. Show energy savings vs. PID baseline and seasonal parameter adaptation of $\hat{P}$.

§11 • Phase 8 — Tests

11.1 Test: Lyapunov = Value Function Identity

(`tests/test_identity_lyapunov_value.py`)

```

"""
Verify Identity 1: Kleinman's algorithm converges to the CARE solution,
and that the IRL critic converges to the same P given trajectory data.
"""
def test_kleinman_converges_to_care():
    """Kleinman policy iteration on a known 2x2 system should match scipy CARE."""
    ...

def test_irl_critic_converges_to_lyapunov_P():
    """Given trajectory data from a linear system, the IRL Bellman loss
    should drive  $\hat{P} \rightarrow P_{\text{CARE}}$  within tolerance after sufficient episodes."""
    ...

def test_quadratic_basis_reconstructs_P():
    """extract_P() on a QuadraticCritic with known W_c should return the correct
    ...

```

11.2 Test: Costate Identity (tests/test_identity_costate.py)

```

def test_costate_equals_grad_V():
    """lambda_hat from CostateHead should equal torch.autograd.grad(V, e)."""
    ...

def test_optimal_control_equals_lqr_gain():
    """ $u^* = -R^{-1}B^T \nabla V$  should equal  $-K_{\text{LQR}} e$  when  $\hat{V} = e^T P_{\text{LQR}} e$ ."""
    ...

```

11.3 Test: CBF Safety Filter (tests/test_safety.py)

```

def test_cbf_projects_unsafe_control():
    """When u_nom would drive the error outside the ellipsoid, u_safe should sati
    ...

def test_cbf_identity_when_safe():
    """When the nominal control is already safe ( $a \geq 0$ ), u_safe == u_nom exactly."""
    ...

def test_cbf_forward_invariance():
    """Simulate 100 steps starting inside the safe set; should never leave it."""
    ...

```

11.4 Test: Port-Hamiltonian Decoder (tests/test_models.py)

```

def test_dissipation_matrix_psd():
    """ $R_\theta = L^T L$  must be positive semi-definite for any input."""
    ...

def test_J_skew_symmetric():
    """ $J$  should satisfy  $J + J^T = 0$  for all inputs."""
    ...

def test_hamiltonian_positive():
    """ $H_\theta(q, p) > 0$  for all  $(q, p)$  (enforced by softplus output)."""
    ...

```

11.5 Test: IRL Loss (`tests/test_irl.py`)

```

def test_irl_bellman_error_zero_at_true_value():
    """Given trajectories of the true linear system and the true  $P$  matrix,
    the IRL Bellman error should be  $\approx 0$  (up to numerical integration error)."""
    ...

def test_irl_loss_decreases_with_correct_update():
    """One gradient step on  $L_{\text{IRL}}$  with a suboptimal critic should reduce the Bell
    ...

```

11.6 Smoke Tests (`tests/test_smoke.py`)

```

def test_full_forward_pass_no_crash():
    """Instantiate default config, run 10 steps of RealtimeInferenceEngine, no ex
    ...

def test_pretrain_one_epoch():
    """Run one epoch of pre-training on random data; loss should be finite."""
    ...

def test_cotrain_one_episode():
    """Run one episode of co-training; all loss terms should be finite scalars."""
    ...

```

§12 · Phase 9 — CI/CD

Create `.github/workflows/ci.yml`:

```

name: CI

on:
  push:
    branches: [master, main, develop]
  pull_request:
    branches: [master, main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.9", "3.10", "3.11"]

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v4
        with:
          python-version: ${ matrix.python-version }

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Lint with flake8
        run: flake8 src/ tests/ --max-line-length=100 --ignore=E203,W503

      - name: Type check with mypy
        run: mypy src/pits_mras/ --ignore-missing-imports

      - name: Run tests with coverage
        run: pytest tests/ -v --cov=pits_mras --cov-report=xml --cov-report=term-

      - name: Upload coverage report
        uses: codecov/codecov-action@v3
        with:
          file: coverage.xml

```

Also create `pyproject.toml` **for modern tooling:**

```

[tool.black]
line-length = 100

```

```
target-version = ["py39", "py310", "py311"]
```

```
[tool.isort]  
profile = "black"  
line_length = 100
```

```
[tool.mypy]  
python_version = "3.9"  
warn_return_any = true  
warn_unused_configs = true  
ignore_missing_imports = true
```

```
[tool.pytest.ini_options]  
testpaths = ["tests"]  
addopts = "-v --tb=short"
```

§13 · Implementation Order and Acceptance Criteria

Execute phases in strict order. Each phase has a verifiable acceptance criterion before proceeding to the next.

Phase 1 (Foundation): `python -c "from pits_mras.utils.lyapunov import solve_lyapunov; import numpy as np; A = -np.eye(2); Q = np.eye(2); P = solve_lyapunov(A, Q); print(P)"` should print `[[0.5, 0], [0, 0.5]]` (since $A=-I$, $P=\frac{1}{2}I$ satisfies $-P + (-P) = -I$).

Phase 2 (Models): `pytest tests/test_models.py -v` — all model unit tests pass.

Phase 3 (Losses): `pytest tests/test_irl.py tests/test_identity_costate.py -v`.

Phase 4 (Controllers): `pytest tests/test_safety.py tests/test_identity_costate.py -v`.

Phase 5 (Training): `pytest tests/test_smoke.py -v` — smoke tests pass with no NaN losses.

Phase 6 (Inference): Run `examples/robotic_manipulator.py` for 100 steps; plots are generated without error.

Phase 7–9: All tests pass in CI; `pytest --cov=pits_mras` reports $\geq 60\%$ coverage.

§14 · Additional Notes for Claude Code

Tensor shape conventions: All batched tensors should follow PyTorch convention `[batch, dim]` or `[batch, T, dim]`. Never `[dim, batch]`. Always add `.unsqueeze(0)` when converting single-sample inference to batched.

Device handling: Every `nn.Module` that stores tensors via `register_buffer` will automatically move them to the correct device with `.to(device)`. Do not hardcode `torch.cuda` anywhere — always use `device = cfg.training.device`.

Autograd requirements: The costate computation `torch.autograd.grad(V, e)` requires `e.requires_grad_(True)` before calling the critic. In inference, call with `torch.no_grad()` for everything except the costate head (which needs grad). Use `create_graph=True` when the gradient itself will be differentiated (e.g., in the adjoint-dynamics loss `L_adjoint`).

Numerical stability: The quadratic basis $\phi(e)$ can have very large values for large $\|e\|$. Normalize the inputs before passing to the critic. The `PortHamiltonianDecoder` uses `torch.autograd.grad` on `H_0`; always ensure `qp.requires_grad_(True)` is set before the Hamiltonian evaluation.

Mathematical bibliography: When implementing any component, the key references are:

- IRL critic (Loss §6.4, Controller §7.3): Vrabie & Lewis 2009, DOI 10.1016/j.neunet.2009.03.008
- Policy iteration / Kleinman: IEEE TAC AC-13(1):114–115, 1968. DOI 10.1109/TAC.1968.1098829
- CBF safety filter: Ames et al. 2017, arXiv:1609.06408
- DPG actor update: Silver et al. ICML 2014, PMLR 32(1):387–395
- HJB PINN loss: Wang & Wu, AIChE J. 70(10):e18542, 2024. DOI 10.1002/aic.18542
- Port-Hamiltonian RL: Sprangers et al., arXiv:1212.5524